

Spring Boot First Application

(Compared with Spring Core)

Goal

Understand how Spring Boot makes development easier compared to Spring Core.

We will compare:

- Spring Core (Java Configuration)
- Spring Boot (Auto Configuration)

Project Directory Structure

This is the Eclipse Project Explorer view of the project, shown here before the code so the folder and package layout is clear from the start.

```
Demo004 [boot] [Downloads master]
├─ src/main/java
│   ├─ com.example.demo
│   │   ├─ com.example.demo.FirstApp
│   │   │   ├─ Robot.java
│   │   │   └─ Sensor.java
│   │   └─ Demo004Application.java
│   └─ com.example.demo.FirstApp
│       └─ com.nit.day60
├─ src/main/resources
│   └─ application.properties
├─ src/test/java
├─ JRE System Library [JavaSE-17]
├─ Maven Dependencies
├─ src
├─ target
├─ HELP.md
├─ mvnw
├─ mvnw.cmd
└─ pom.xml
```

Spring Core vs Spring Boot

Spring Core	Spring Boot
Need to create a <code>@Configuration</code> class.	No need to create a separate configuration class.
Write <code>@ComponentScan</code> manually.	Done automatically by <code>@SpringBootApplication</code> .
Write <code>@PropertySource</code> manually.	<code>application.properties</code> loads automatically.
Create IOC Container using <code>AnnotationConfigApplicationContext</code> .	Create IOC Container using <code>SpringApplication.run()</code> .
More configuration.	Less configuration.

Memory Tip: Spring Core asks us to configure many things manually. Spring Boot does most of those configurations automatically.

Spring Core Java Configuration

In Spring Core we usually write:

```
@Configuration
@ComponentScan("com.demo")
@PropertySource("classpath:application.properties")
public class AppConfig{
}
```

Then we create the IOC Container manually.

```
ApplicationContext context =
    new AnnotationConfigApplicationContext(AppConfig.class);
```

Here we explicitly tell Spring which configuration class should be used.

Spring Boot Approach

In Spring Boot we simply write:

```
@SpringBootApplication
public class Demo004Application{
}
```

No separate configuration class is required.

Spring Boot automatically performs most common configurations.

@SpringBootApplication

`@SpringBootApplication` is a combination of three annotations.

- `@SpringBootConfiguration` — Works like `@Configuration`. This class becomes the Java Configuration class.
- `@EnableAutoConfiguration` — Automatically configures many Spring features based on project dependencies.
- `@ComponentScan` — Automatically scans the current package and its sub-packages for Spring Beans.

How `SpringApplication.run()` Works

```
ApplicationContext context =
    SpringApplication.run(Demo004Application.class, args);
```

This single line performs multiple tasks.

- Starts the Spring Boot application.
- Creates the IOC Container (`ApplicationContext`).
- Performs Component Scanning.
- Creates all required beans.
- Loads `application.properties` automatically.
- Returns the `ApplicationContext` object.

Memory Tip: In Spring Core we create the container manually. In Spring Boot `SpringApplication.run()` creates and returns the container for us.

Understanding Our Project

Package Structure

```
com.example.demo
    Demo004Application

com.example.demo.FirstApp
    Robot
    Sensor
```

`Demo004Application` is inside the parent package, so `@ComponentScan` automatically scans `FirstApp`.

Understanding the Sensor Class

```
@Component
public class Sensor{
}
```

Sensor is our own class.

We added `@Component`, so Spring automatically creates its object.

Reading values

```
@Value("${sensorActivate}")
private boolean isSensorActivate;

@Value("${chipName}")
private String chipName;
```

These values come from `application.properties`.

Example:

```
sensorActivate=true
chipName=Intel
```

Spring automatically injects these values.

Understanding the Robot Class

Notice carefully.

Robot has no `@Component` annotation.

```
public class Robot{
}
```

So Spring will not create its object automatically.

We create it manually using `@Bean`.

Creating Robot Bean

```
@Bean
public Robot robot(){
    Robot robot=new Robot();

    robot.setRoboCompany(environment.getProperty("companyName"));
    robot.setMotorPower(environment.getProperty("motorPower"));

    return robot;
}
```

Here Spring executes this method and registers the returned object as a Bean.

Bean name becomes robot (method name).

Why Robot Uses @Bean Instead of @Component

There are two situations.

Situation 1 – We can modify the class

```
@Component
public class Sensor{
}
```

Spring creates the object automatically.

Situation 2 – We cannot or do not want to modify the class

```
@Bean
public Robot robot(){
    return new Robot();
}
```

We create the object inside a @Bean method.

Interview Point: @Component is used when we can annotate the class. @Bean is used for third-party classes, existing library classes, or classes where we don't want to place Spring annotations.

Environment Object

```
@Autowired
Environment environment;
```

Environment helps us read values from application.properties.

Example:

```
environment.getProperty("companyName");
```

This reads:

```
companyName=Tesla
```

Autowiring Inside Robot

```
@Autowired
private Sensor sensor;
```

Spring finds the Sensor Bean and injects it into Robot.

We never write:

```
new Sensor();
```

Spring performs the dependency injection automatically.

Execution Flow

- JVM starts the main() method.
- SpringApplication.run() starts Spring Boot.
- IOC Container is created.
- @ComponentScan scans packages.
- Sensor Bean is created.
- application.properties is loaded.
- robot() method executes.
- Company and Motor Power values are injected.
- Sensor is injected into Robot.
- Robot Bean is stored inside IOC Container.
- context.getBean("robot") returns the object.
- toString() prints the complete object.

Complete Bean Flow

This is exactly what happens inside your project.

Practical Difference Students Should Remember

Question	Spring Core	Spring Boot
Configuration class	Required	Not required separately
Component Scan	Manual	Automatic
Property Source	Manual	Automatic
IOC Container	AnnotationConfigApplicationContext	SpringApplication.run()
Bean creation	@Bean / @Component	Same, but easier configuration

Interview Questions

Q1. How many annotations are present inside @SpringBootApplication?

Answer: Three.

- @SpringBootConfiguration
- @EnableAutoConfiguration
- @ComponentScan

Q2. What does SpringApplication.run() do?

Answer: It starts the Spring Boot application, creates the IOC Container (ApplicationContext), performs component scanning, loads properties, creates beans, and returns the ApplicationContext.

Q3. When do we use @Component?

Answer: When we can modify the class and want Spring to create the bean automatically.

Q4. When do we use @Bean?

Answer: When we cannot or do not want to place Spring annotations inside the class, such as third-party or existing classes.

Full Code for Practice

The complete, ready-to-run code for all three classes discussed above is given here so it can be used directly for practice.

Robot.java

```
package com.example.demo.FirstApp;
/*
 * No need of creating any java configuration class
 * |- @Configuration
 * |- no need of writing any configuration annotation
 * inside any class
 *
 */
import org.springframework.beans.factory.annotation.Autowired;

public class Robot {
    private String roboCompany;
    private String motorPower;
    @Autowired
    private Sensor sensor;

    public Robot() {
        super();
    }

    public String getRoboCompany() {
```

```

        return roboCompany;
    }

    public void setRoboCompany(String roboCompany) {
        this.roboCompany = roboCompany;
    }

    public String getMotorPower() {
        return motorPower;
    }

    public void setMotorPower(String motorPower) {
        this.motorPower = motorPower;
    }

    public Sensor getSensor() {
        return sensor;
    }

    public void setSensor(Sensor sensor) {
        this.sensor = sensor;
    }

    @Override
    public String toString() {
        return "Robot [roboCompany=" + roboCompany + ", motorPower=" + motorPower
+ ", sensor=" + sensor + "]";
    }
}

```

Sensor.java

```

package com.example.demo.FirstApp;

import org.springframework.beans.factory.annotation.Value;
import org.springframework.context.annotation.PropertySource;
import org.springframework.stereotype.Component;

@Component
public class Sensor {

    @Value("${sensorActivate}")
    private boolean isSensorActivate;
    @Value("${chipName}")
    private String chipName;
}

```



```

    public Sensor() {
        super();
    }

    public boolean isSensorActivate() {
        return isSensorActivate;
    }

    public void setSensorActivate(boolean isSensorActivate) {
        this.isSensorActivate = isSensorActivate;
    }

    public String getChipName() {
        return chipName;
    }

    public void setChipName(String chipName) {
        this.chipName = chipName;
    }

    @Override
    public String toString() {
        return "Sensor [isSensorActivate=" + isSensorActivate + ", chipName=" +
chipName + "]";
    }
}

```

Demo004Application.java

```

package com.example.demo;

import org.springframework.beans.factory.annotation.Autowired;
import org.springframework.boot.SpringApplication;
import org.springframework.boot.autoconfigure.SpringBootApplication;
import org.springframework.context.ApplicationContext;
import org.springframework.context.annotation.AnnotationConfigApplicationContext;
import org.springframework.context.annotation.Bean;
import org.springframework.core.env.Environment;

import com.example.demo.FirstApp.Robot;

@SpringBootApplication

public class Demo004Application {

```

```

@Autowired
Environment environment;

@Bean // 4
public Robot robot() {
    Robot robot = new Robot();
    robot.setRoboCompany(environment.getProperty("companyName"));
    robot.setMotorPower(environment.getProperty("motorPower"));

    return robot;
}

// Step-1
public static void main(String[] args) {
    ApplicationContext context =
        // Step- II
        SpringApplication.run(Demo004Application.class, args);
    // ApplicationContext context=new AnnotationConfigApplicationContext();
    Robot bean = context.getBean("robot", Robot.class);
    System.out.println(bean);

}

}

```